Effect Systems & Interpreters

Effect Interpreters · Describe IO report, run at edge · Build with Waqar

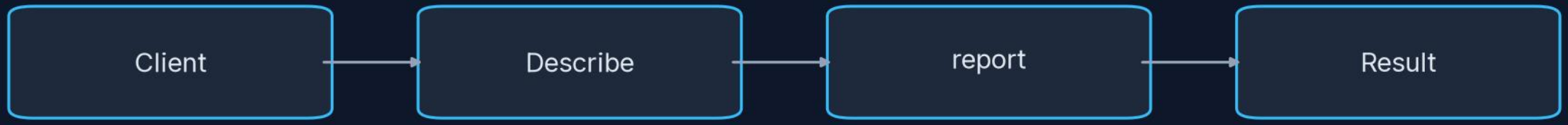
Today's Hook

- Problem: Describe IO report, run at edge
- Pattern: Effect Interpreters — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Describe IO report, run at edge
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Effect Interpreters: Describe IO report, run at edge

Real-World Scenario

- Running example: Describe IO report, run at edge
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.io;

import com.buildwithwaqar.patterns.functional.io.IO;
import java.util.concurrent.atomic.AtomicInteger;
/** Problem: describe a side-effecting report pipeline; run only at the edge. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== IO / Effect Interpreter problem ===");
    var hits = new AtomicInteger();
    IO<String> report = IO.delay(() -> { hits.incrementAndGet(); return "rows"; })
        .map(s -> s + "=" + 42);
    System.out.println("described hits=" + hits.get());
    System.out.println("run=" + report.unsafeRun() + " hits=" + hits.get());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.io;

import java.util.function.Function;
import java.util.function.Supplier;

/** Tiny IO effect — suspend side effects, interpret at the edge. */
public final class IO<A> {
    private final Supplier<A> thunk;

    private IO(Supplier<A> thunk) { this.thunk = thunk; }

    public static <A> IO<A> delay(Supplier<A> thunk) { return new IO<>(thunk); }
    public static <A> IO<A> pure(A a) { return new IO<>(() -> a); }

    public <B> IO<B> map(Function<? super A, ? extends B> f) {
        return new IO<>(() -> f.apply(thunk.get()));
    }

    public <B> IO<B> flatMap(Function<? super A, IO<B>> f) {
        return new IO<>(() -> f.apply(thunk.get()).unsafeRun());
    }
```

Solution (2)

```
    /** Unsafe: runs effects. Call only at the program edge. */
    public A unsafeRun() { return thunk.get(); }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.io;

import com.buildwithwaqar.patterns.functional.io.IO;
import java.util.concurrent.atomic.AtomicInteger;
/** Problem: describe a side-effecting report pipeline; run only at the edge. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== IO / Effect Interpreter problem ===");
    var hits = new AtomicInteger();
    IO<String> report = IO.delay(() -> { hits.incrementAndGet(); return "rows"; })
        .map(s -> s + "=" + 42);
    System.out.println("described hits=" + hits.get());
    System.out.println("run=" + report.unsafeRun() + " hits=" + hits.get());
  }
}
```

Walkthrough Output

```
=== IO / Effect Interpreter problem ===
described hits=0
run=rows=42 hits=1
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Effect Interpreters.

Related Patterns

- Contrast Effect Interpreters with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Effect Interpreters → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.